

Ricorsione

Metodo alternativo alla costruzione di procedure iterative

Procedura ricorsiva se è definita in funzione di se stessa

↓
all'interno della procedura
esiste una chiamata
alla procedura stessa

Fattoriale

```
int fatt(int n) {  
    if (n ≤ 1) return n;  
    else return n * fatt(n-1);  
}
```

$$\text{fatt}(n) = \begin{cases} n & n \leq 1 \\ n * \text{fatt}(n-1) & \text{altrimenti} \end{cases}$$

Attenzione!

Le procedure che chiamano se stesse senza cambiare il parametro non soddisfano le proprietà di ricorsione.

Ricorsione vs iterazione

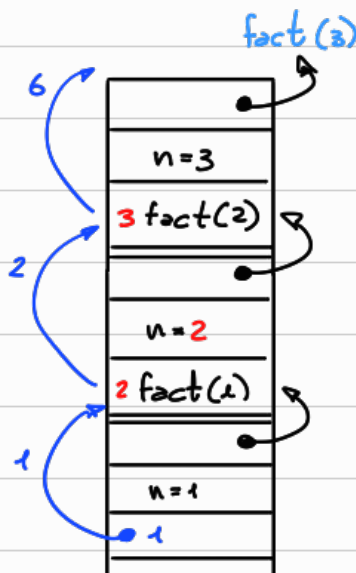
↓
possibile se le procedure
possono essere definite
in termini di se stesse

Lo sempre possibile in qualunque
linguaggio turing completo

↓ ↓
Allocazione Ricorsione
dinamica in coda

→ ogni programma iterativo può essere riscritto in modo ricorsivo e viceversa.

```
int fact(int n) {  
    if (n ≤ 1) return n;  
    else return n * fact(n-1);  
}
```



Ricorsione in coda

Chiama in coda :

- f chiama g

- la chiamata di g è detta in coda se f restituisce il valore restituito da g senza ulteriori computazioni.

f è ricorsiva in coda : se la chiamata ricorsiva a se stessa è in coda

```
int fact (int n) {  
    fact-rc (n, 1);  
}
```

valore iniziale per res → indicativamente è il risultato del caso base

```
int fact-rc (int n, int res) {  
    if (n==1) return res;  
    else fact-rc (n-1, n*res);  
}
```

// abbiamo spostato il calcolo all'interno della chiamata di funzione

$fact(3) \rightarrow fact-rc(3, 1) \rightarrow fact-rc(2, 3) \rightarrow fact-rc(1, 6)$

[0:36:00]

Fibonacci

$$\begin{cases} fib(0) = 1 \\ fib(1) = 1 \\ fib(n) = fib(n-1) + fib(n-2) & n \geq 2 \end{cases}$$

```
int fib (int n) {  
    if (n <= 1) return n;  
    else return fib(n-1) + fib(n-2);  
}
```

→ complessità esponenziale sia in tempo che in spazio

```
int fib(int n) {
    fib-rc(n, 1, 1);
}
```

base per $n=0$
base per $n=1$
vanno inizializzati a valori che permettono di restituire subito il risultato del caso base

```
int fib-rc(int n, int res1, int res2) {
    if (n <= 1) return res2;
    else return fib-rc(n-1, res2, res1+res2);
}
```

risultato al passo precedente risultato calcolato

[0:48:34]

Esercizio ricorsione

Chiede di trasformare una procedura ricorsiva data in una procedura ricorsiva in coda.

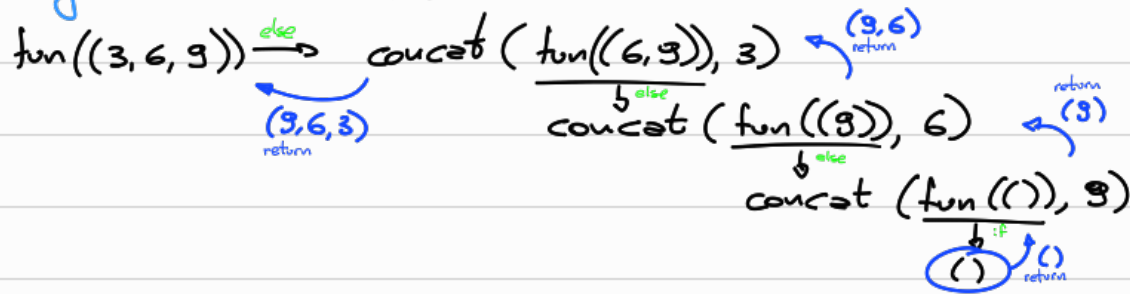
- def. breve di ricorsione
- def. breve di ricorsione in coda
- esecuzione della procedura data sull'input specificato
- breve descrizione a parole di cosa fa la procedura, ovvero come trasforma l'input
- scrittura della procedura ricorsiva in coda con chiamata minimale (minimizzazione di res)
- esecuzione della procedura scritta sull'input specificato

Prog. ricorsivo dato:

```
lista fun(lista list) {
    if (length(list) == 0) return list;
    else return concat(fun(cdr(list)), car(list));
}
```

lunghezza coda della lista testa della lista
concatenazione restituisce una lista restituisce un elemento

eseguire su (3,6,3)



=> Il programma esegue una reverse della lista

Dobbiamo fornire la versione ricorsiva in coda

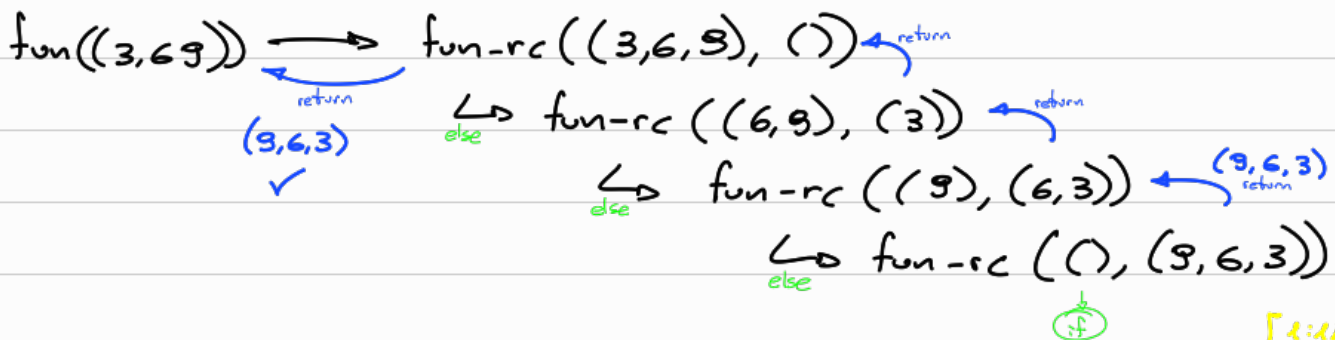
```

lista fun (lista list) {
  fun-rc (list, ());
}

lista fun-rc (lista list, lista res) {
  if (length (list) == 0) return res;
  else return fun-rc (cdr (list), concat (car (list), res));
}

```

Esecuzione su (3,6,3)



[4:44:00]

Esempio ricorsione: 2

```

int fun (lista list) {
  if (length (list) == 0) return 0;
  else return (car (list)) + fun (cdr (list));
}

```

Esecuzione su $(3, 6, 9)$

$$\text{fun}((3, 6, 9)) \rightarrow 3 + \frac{\text{fun}((6, 9))}{\text{return } 15} = 18$$

$$\text{fun}((6, 9)) \rightarrow 6 + \frac{\text{fun}((9))}{\text{return } 9} = 15$$

$$\text{fun}((9)) \rightarrow 9 + \frac{\text{fun}(())}{\text{return } 0} = 9$$

=> Calcola la somma degli elementi che compongono la lista

Procedura ricorsiva in code

```
int fun (list< list>) {  
    return fun-rc(list, 0);  
}  
  
int fun-rc (list< list>, int res) {  
    if (length(list) == 0) return res;  
    else return fun-rc( cdr(list), res + car(list));  
}
```

deve essere del tipo
del valore di ritorno!

Esecuzione su $(3, 6, 9)$

$$\text{fun}((3, 6, 9)) \rightarrow \text{fun-rc}((3, 6, 9), 0)$$

$$\text{else } \text{fun-rc}((6, 9), 0+3)$$

$$\text{else } \text{fun-rc}((9), 6+3)$$

$$\text{else } \text{fun-rc}(), 9+9$$

Esercizio da fare

```

list* fun (list* bst) {
    if (length(list) == 0) return ();
    else return concat(2 * car(list), fun(cdr(list)));
}

```

Eseguire su $(3, 6, 9)$

